

149. Rotate a Linked List

Problem Statement

Given the head of a singly linked list and an integer k , rotate the linked list to the right by k places.

Each rotation moves the last node of the linked list to the beginning.

Example

Input:

head = [1, 2, 3, 4, 5]

$k = 2$

Output:

[4, 5, 1, 2, 3]

Explanation:

After the first rotation:

[5, 1, 2, 3, 4]

After the second rotation:

[4, 5, 1, 2, 3]

Brute Force Approach

Intuition

The idea is to rotate the linked list one position to the right, k times.

In each rotation:

1. Traverse the linked list to find the last node and the second-last node.
2. Disconnect the last node from the list.
3. Make the last node the new head.
4. Connect the last node to the previous head.

Repeat this process k times.

Java Code

```
class Solution {
```

```

public ListNode rotateRight(ListNode head, int k) {
    if (head == null || head.next == null || k == 0) {
        return head;
    }

    while (k > 0) {
        ListNode curr = head;

        while (curr.next.next != null) {
            curr = curr.next;
        }

        ListNode last = curr.next;
        curr.next = null;

        last.next = head;
        head = last;

        k--;
    }

    return head;
}

```



Complexity Analysis

- **Time Complexity:** $O(N \times K)$ — Each rotation requires traversing the linked list.
- **Space Complexity:** $O(1)$ — No additional data structures are used.



Limitation

This approach becomes inefficient when k is very large.

Optimal Approach

Intuition

Instead of rotating the list one step at a time, we can use the length of the linked list to reduce the number of operations.

If the linked list has N nodes, rotating it N times returns the original list. Therefore, we only need:

$$k = k \% N$$

Algorithm

1. If the list is empty or contains only one node, return the head.
2. Traverse the list to find its length and last node.
3. Update k using $k \% \text{length}$.
4. If $k == 0$, return the original list.
5. Connect the last node to the head to form a circular linked list.
6. Find the new tail at position $\text{length} - k$.
7. Set the node after the new tail as the new head.
8. Break the circular link and return the new head.

Java Code

```
class Solution {  
  
    public ListNode rotateRight(ListNode head, int k) {  
        if (head == null || head.next == null || k == 0) {  
            return head;  
        }  
  
        ListNode tail = head;  
        int length = 1;
```

```
// Find the length and tail
while (tail.next != null) {
    tail = tail.next;
    length++;
}

// Reduce unnecessary rotations
k = k % length;

if (k == 0) {
    return head;
}

// Make the list circular
tail.next = head;

// Find the new tail
int stepsToNewTail = length - k;
ListNode newTail = head;

for (int i = 1; i < stepsToNewTail; i++) {
    newTail = newTail.next;
}

// Update the head and break the circle
ListNode newHead = newTail.next;
newTail.next = null;
```

```
        return newHead;
    }
}
```

Complexity Analysis

- **Time Complexity:** $O(N)$ — The linked list is traversed a constant number of times.
 - **Space Complexity:** $O(1)$ — Only a few pointer variables are used.
-

Approach Comparison

Criteria	Brute Force	Optimal
Technique	Rotate one step at a time	Length and circular list
Time Complexity	$O(N \times K)$	$O(N)$
Space Complexity	$O(1)$	$O(1)$
Efficiency	Slow for large K	Efficient

Key Concepts Learned

- Singly Linked List
- Pointer Manipulation
- Circular Linked List
- Modulo Operation (%)
- Edge Case Handling
- Time and Space Complexity Analysis

Conclusion

The brute force approach rotates the list one step at a time, while the optimal approach reduces unnecessary rotations using the list length and temporarily forms a circular linked list.

The optimal approach is preferred because it achieves $O(N)$ time and $O(1)$ auxiliary space.

Tags

Java Linked List DSA Brute Force Optimal Approach Pointer Manipulation

